Unit 3: Input/Output Functions
Teaching Focus: Concept + Syntax + Program Implementation + Practical Understanding

---

1. Learning Objectives

After completing this unit, students should be able to:

1. Explain the concept of input and output in C.
2. Distinguish between **console I/O and file I/O**.
3. Differentiate **formatted and unformatted I/O**.
4. Use printf() for formatted output.
5. Use scanf() for formatted input.
6. Explain the role of the **address operator &** in scanf().
7. Use appropriate **format specifiers** for different data types.
8. Apply field width, alignment and precision.
9. Explain and use character I/O functions such as getchar() and putchar().
10. Explain string input/output using fgets() and puts().
11. Explain escape sequences.
12. Use sprintf() and sscanf() for processing data in strings.
13. Identify common input-related errors in C programs.
14. Develop small programs using appropriate I/O functions.

---

2. Introduction to Input/Output in C

A computer program generally performs three fundamental activities:
Input → Processing → Output
For example:

INPUT
↓
Student marks
↓
PROCESS
↓
Calculate total
↓
OUTPUT
↓
Display result

Consider:

```
#include <stdio.h>

int main()
{
   int marks;

   printf("Enter marks: ");
   scanf("%d", &marks);

   printf("Your marks = %d", marks);

   return 0;
}
```

Here:

- scanf() receives input.

- The variable marks stores the input.
- printf() displays the result.

---

3. Does C Have Input/Output Keywords?

A common beginner misconception is:
"printf() and scanf() are C keywords."
They are not.
C does not have printf or scanf as language keywords.
They are **library functions** provided through the standard I/O library.
For example:
#include <stdio.h>
The header file <stdio.h> provides declarations for many standard input/output functions.
Some important functions include:
printf()
scanf()
getchar()
putchar()
puts()
fgets()
sprintf()
sscanf()

---

4. What is <stdio.h>?

stdio.h stands for:
Standard Input Output Header
It contains declarations related to standard input/output operations.
Example:

```
#include <stdio.h>

int main()
{
   printf("Hello BIT Students");

   return 0;
}
```

Without including the appropriate declaration for printf(), the compiler may not correctly know the function's interface.

---

5. Types of Input/Output

I/O can broadly be divided into:
A. Console I/O
Communication between the program and:

- Keyboard
- Monitor/screen

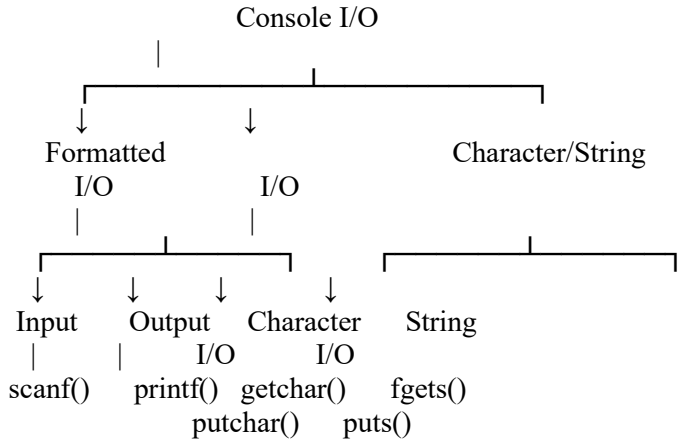
For example:
scanf("%d", &age);
printf("%d", age);
B. File I/O
Reading data from or writing data to files.
For example:
Program

↓
File
↓
student.txt
File I/O is normally covered in greater detail in a later unit.

---

6. Classification of Console I/O Functions

Console I/O functions can be broadly classified as:



Important note

Older C textbooks often classify functions such as getch(), getche(), and gets() under unformatted I/O.

However, these functions are **not standard ISO C functions**.

For modern C programming, students should primarily learn:

- getchar()
- putchar()
- fgets()
- puts()

---

7. Formatted I/O

Formatted I/O allows us to specify **how data should be interpreted or displayed**.

The two most important functions are:

printf()

scanf()

---

8. printf() Function

Definition

printf() is a formatted output function used to display information on the screen.

Syntax

printf("format string", arguments);

Example:

printf("Hello World");

Output:

Hello World

Example 1: Display an integer

```
#include <stdio.h>

int main()
{
   int age = 20;

   printf("Age = %d", age);

   return 0;
}
```

Output:
Age = 20
Here:
%d
is the conversion specification for a signed decimal integer.

---

9. printf() with Multiple Variables

```
#include <stdio.h>

int main()
{
   int age = 20;
   float height = 5.8;
   char grade = 'A';

   printf("Age = %d\n", age);
   printf("Height = %.1f\n", height);
   printf("Grade = %c\n", grade);

   return 0;
}
```

Output:
Age = 20
Height = 5.8
Grade = A

---

10. How printf() Works

Consider:
printf("Age = %d", age);
printf() processes the format string from left to right.
"Age = " → printed directly

%d → replaced by value of age
Therefore, if:
age = 25;
the output becomes:
Age = 25

---

11. Escape Sequences

Escape sequences are special character combinations beginning with \.

For example:
printf("Hello\nWorld");
Output:
Hello
World
The sequence:
\n
means newline.

Important Escape Sequences

Escape	Meaning
\n	New line
\t	Horizontal tab
\b	Backspace
\r	Carriage return
\a	Alert/bell
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\0	Null character

Example
printf("Name:\tMahesh\nAge:\t48");
Output:
Name: Mahesh
Age: 48

---

12. scanf() Function
scanf() is a formatted input function.
It receives data from **standard input**, normally the keyboard.
Syntax
scanf("format string", &variable);
Example:
int age;

scanf("%d", &age);

---

13. Why Do We Use & with scanf()?
This is one of the most important concepts in C.
Suppose:
int age;
The variable age has a memory location.
Conceptually:
Variable Memory Address Value

age 1000 ?
When we write:
scanf("%d", &age);
&age means:
Address of age
Therefore, scanf() knows **where to store the input value**.

For example, if the user enters:
25
then conceptually:
scanf()
↓
address of age
↓
store 25
↓
age = 25

---

14. Why Not scanf("%d", age)?

Incorrect:
scanf("%d", age);
Correct:
scanf("%d", &age);
For an ordinary non-pointer variable, forgetting & can lead to undefined behavior and may crash the program.

---

15. Exception: Character Arrays / Strings

There is an important exception.
For a character array:
char name[30];
scanf("%29s", name);
We normally **do not write &name**.
Why?
Because the array name generally converts to a pointer to its first element when passed to a function.

---

16. Taking Multiple Inputs

```
#include <stdio.h>

int main()
{
   int a, b;

   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);

   printf("A = %d\n", a);
   printf("B = %d\n", b);

   return 0;
}
```

Input:
10 20
Output:
A = 10
B = 20
The user can also enter:
10
20

because whitespace separates the values.

---

17. Whitespace in scanf()

The following can separate input values:

- Space
- Tab
- Newline

For:
scanf("%d %d", &a, &b);
all of these are generally acceptable:
10 20
or:
10 20
or:
10
20

---

18. Reading Different Data Types

Integer

```
int age;

scanf("%d", &age);
```

Float

```
float salary;

scanf("%f", &salary);
```

Double

```
double price;

scanf("%lf", &price);
```

Character

```
char gender;

scanf(" %c", &gender);
```

Notice the space before %c.
It can help skip whitespace left in the input stream.

---

19. Format Specifiers

Format specifiers tell the I/O function what type of data is being processed.

Integer Types

Specifier	Meaning
%d	Signed decimal integer
%i	Integer
%u	Unsigned integer
%ld	Long signed integer
%lu	Long unsigned integer
%o	Octal
%x	Hexadecimal

Example:

```
int n = 100;

printf("%d", n);
```

---

20. Floating-Point Format Specifiers

Specifier	Typical use
%f	float with printf
%f	float * with scanf
%lf	double * with scanf
%Lf	long double

Example:

```
float temperature = 25.5;

printf("%.2f", temperature);
```

Output:

```
25.50
```

---

21. Character and String Format Specifiers

Character

%c

Example:

```
char grade = 'A';

printf("%c", grade);
```

String

%s

Example:

```
char name[] = "Mahesh";

printf("%s", name);
```

Output:

```
Mahesh
```

---

22. Important Format-Specifier Table

Data type	printf()	scanf()
int	%d	%d
unsigned int	%u	%u
long int	%ld	%ld
float	%f	%f
double	%f	%lf
long double	%Lf	%Lf
char	%c	%c
string	%s	%s

Important exam point

Students frequently confuse:

```
printf("%f", double_variable);
```

with:

scanf("%lf", &double_variable);
For scanf(), a double requires:
%lf

---

23. Example: Student Information Program

```
#include <stdio.h>

int main()
{
   int roll;
   char name[30];
   float marks;

   printf("Enter roll number: ");
   scanf("%d", &roll);

   printf("Enter name: ");
   scanf("%29s", name);

   printf("Enter marks: ");
   scanf("%f", &marks);

   printf("\n--- Student Information ---\n");
   printf("Roll No : %d\n", roll);
   printf("Name    : %s\n", name);
   printf("Marks   : %.2f\n", marks);

   return 0;
}
```

---

24. Problem with %s

Consider:
char name[30];

scanf("%29s", name);
If the user enters:
Ram Kumar
only:
Ram
will normally be read.
Why?
Because %s stops at whitespace.
Therefore:
Ram Kumar
  ↑
scanf stops here
This is a very important limitation.

---

25. Better Method for Reading a Full Line

For modern C programming, use:
fgets()

Example:

```
#include <stdio.h>

int main()
{
   char name[50];

   printf("Enter your full name: ");
   fgets(name, sizeof(name), stdin);

   printf("Name = %s", name);

   return 0;
}
```

Input:
Ram Kumar Sharma
Output:
Name = Ram Kumar Sharma

---

26. fgets() vs gets()

Older textbooks may teach:

```
gets(name);
```

Do not use it in modern C.

gets() does not know the size of the destination buffer. Therefore, excessive input can overwrite memory.

Example of unsafe code:

```
char name[10];

gets(name);
```

If the user enters more than 9 characters plus the terminating null character, memory may be overwritten.

Modern replacement

Use:

```
fgets(name, sizeof(name), stdin);
```

This provides a maximum number of characters to read.

---

27. puts() Function

puts() displays a string and automatically adds a newline.

Example:

```
#include <stdio.h>

int main()
{
   char message[] = "Welcome to BIT";

   puts(message);

   return 0;
}
```

Output:
Welcome to BIT

Compare:
printf("%s\n", message);
and:
puts(message);
Both produce essentially the same displayed line.

---

28. getchar()
getchar() reads one character from standard input.
Example:

```
#include <stdio.h>

int main()
{
   char ch;

   printf("Enter a character: ");
   ch = getchar();

   printf("You entered: %c", ch);

   return 0;
}
```

If input is:
A
the variable ch receives:
'A'

---

29. putchar()
putchar() outputs a single character.
Example:

```
#include <stdio.h>

int main()
{
   char ch = 'A';

   putchar(ch);

   return 0;
}
```

Output:
A

---

30. Character I/O Summary

Function	Purpose	Standard C?
getchar()	Read one character	Yes
putchar()	Output one character	Yes
fgets()	Read a line/string safely	Yes
puts()	Output a string + newline	Yes

Function	Purpose	Standard C?
getch()	Immediate character input	No
getche()	Immediate echoed character input	No
gets()	Read string unsafely	Removed from C standard

---

31. getch() and getche()

You may encounter these functions in older versions of *Let Us C* and old DOS/Turbo C programs.

getch()

Traditionally:

- reads one character immediately
- does not echo the character
- does not require Enter

Example:

```
ch = getch();
```

getche()

Traditionally:

- reads one character immediately
- echoes the character
- does not require Enter

Example:

```
ch = getche();
```

Important

They belong to non-standard compiler-specific libraries such as old <conio.h> environments.

Do not teach them as standard C functions.

---

32. Field Width

Field width determines the **minimum number of character positions** used for output.

Example:

```
printf("%10d", 25);
```

Conceptually:

```
      25
<-------->
  10 columns
```

The number is right-aligned by default.

---

33. Left Alignment

Use - to left-align the value.

```
printf("%-10d", 25);
```

Conceptually:

```
25
<-------->
10 columns
```

This is useful when creating tables.

Example:

```
printf("%-10s %-10s\n", "Roll", "Marks");
printf("%-10d %-10.2f\n", 1, 85.50);
```

Output:

```
Roll       Marks
1          85.50
```

---

34. Precision

Precision is commonly used with floating-point output.

Example:

```
float pi = 3.1415926;

printf("%.2f", pi);
```

Output:

3.14

Examples:

```
printf("%.1f", pi);
```

Output:

3.1

```
printf("%.3f", pi);
```

Output:

3.142

---

35. Combining Width and Precision

Example:

```
printf("%10.2f", 123.4567);
```

Meaning:

10
↓
minimum field width

2
↓
digits after decimal point

Output is approximately:

```
   123.46
```

---

36. Formatted Student Mark Sheet

```
#include <stdio.h>

int main()
{
   int roll = 101;
   float math = 85.5;
   float programming = 91.25;

   printf("%-10s %-10s\n", "Subject", "Marks");
   printf("%-10s %10.2f\n", "Math", math);
   printf("%-10s %10.2f\n", "C", programming);

   return 0;
}
```

This demonstrates:

- %s
- %f
- field width
- left alignment

- precision

---

37. sprintf()
sprintf() means:
String Print Formatted
Unlike printf(), it does not directly print to the screen.
It formats data into a character array.
Example:

```
#include <stdio.h>

int main()
{
   char result[100];
   int age = 20;

   sprintf(result, "Student age = %d", age);

   printf("%s", result);

   return 0;
}
```

The formatted text is stored in:
result

---

38. Why Use sprintf()?
Suppose we want to construct a message dynamically.

```
char message[100];
char name[] = "Ram";
int age = 21;

sprintf(message, "Name: %s, Age: %d", name, age);
```

Now message contains:
Name: Ram, Age: 21
Modern safety note
For bounded formatting, prefer:
snprintf()
when available, because it allows you to specify the destination buffer size.

---

39. sscanf()
sscanf() means:
String Scan Formatted
It reads formatted data from a string instead of directly from keyboard input.
Example:

```
#include <stdio.h>

int main()
{
   char data[] = "25 85.5";
   int age;
   float marks;
```

```
    sscanf(data, "%d %f", &age, &marks);

    printf("Age = %d\n", age);
    printf("Marks = %.2f\n", marks);

    return 0;
}
```

Output:
Age = 25
Marks = 85.50

---

40. printf() vs sprintf() vs scanf() vs sscanf()

Function	Source	Destination
printf()	Variables	Screen
scanf()	Keyboard	Variables
sprintf()	Variables	String
sscanf()	String	Variables

A useful way to remember:
printf → print to screen
scanf → scan from keyboard
sprintf → print into string
sscanf → scan from string

---

41. Common Input Mistakes
Mistake 1: Forgetting &
Wrong:

```
int age;
scanf("%d", age);
```

Correct:

```
scanf("%d", &age);
```

Mistake 2: Wrong format specifier
Wrong:

```
float salary;

scanf("%d", &salary);
```

Correct:

```
scanf("%f", &salary);
```

Mistake 3: Using %f for double in scanf()
Wrong:

```
double x;

scanf("%f", &x);
```

Correct:

```
scanf("%lf", &x);
```

Mistake 4: Assuming %s reads spaces

```
scanf("%s", name);
```

does not normally read:
Ram Kumar
as one complete string.
Use:
fgets(name, sizeof(name), stdin);
instead.

---

42. scanf() Return Value

An important concept for good programming is that scanf() returns the number of successfully assigned input items.

Example:

```
int age;

if (scanf("%d", &age) == 1)
{
   printf("Valid input");
}
else
{
   printf("Invalid input");
}
```

If the user enters:
25
the return value is 1.
If the user enters:
ABC
the conversion fails.
This is a useful introduction to **input validation**.

---

43. A Complete Example

Problem

Write a C program to input a student's:

- Roll number
- Name
- Marks in three subjects

and display the total and average.

Solution

```
#include <stdio.h>

int main()
{
   int roll;
   char name[50];
   float c, math, english;
   float total, average;

   printf("Enter roll number: ");
   scanf("%d", &roll);

   getchar();  // consume leftover newline
```

```
    printf("Enter student name: ");
    fgets(name, sizeof(name), stdin);

    printf("Enter C marks: ");
    scanf("%f", &c);

    printf("Enter Mathematics marks: ");
    scanf("%f", &math);

    printf("Enter English marks: ");
    scanf("%f", &english);

    total = c + math + english;
    average = total / 3;

    printf("\n========== MARK SHEET ==========\n");
    printf("Roll No : %d\n", roll);
    printf("Name    : %s", name);
    printf("C       : %.2f\n", c);
    printf("Math    : %.2f\n", math);
    printf("English : %.2f\n", english);
    printf("Total   : %.2f\n", total);
    printf("Average : %.2f\n", average);

    return 0;
}
```

This single program demonstrates:

- printf()
- scanf()
- getchar()
- fgets()
- %d
- %f
- %s concept
- %.2f
- \n
- formatted output
- variables
- arithmetic processing

---

44. Important Concept: Input Buffer

Students often become confused when they combine:

scanf()

with:

fgets()

For example:

```
int age;
char name[50];

scanf("%d", &age);
fgets(name, sizeof(name), stdin);
```

After entering:
20
the newline generated by pressing Enter may remain in the input stream.
Then fgets() can read that newline rather than waiting for the name.
A simple teaching example is:

```
scanf("%d", &age);
getchar();
fgets(name, sizeof(name), stdin);
```

However, for larger programs, students should learn **robust input handling** rather than relying on ad-hoc getchar() calls.

---

45. Practical Comparison

Requirement	Recommended function
Display integer	printf()
Display float	printf()
Display character	putchar() / printf()
Display string	puts() / printf()
Read integer	scanf()
Read float	scanf()
Read one character	getchar()
Read full line	fgets()
Format data into string	snprintf()
Parse data from string	sscanf()

---

46. Exam-Oriented Questions

Short Questions

1. What is console I/O?
2. What is formatted I/O?
3. What is the purpose of <stdio.h>?
4. What is a format specifier?
5. What is the purpose of & in scanf()?
6. What is the difference between %f and %lf?
7. What is an escape sequence?
8. What is the purpose of \n?
9. What is field width?
10. What is precision?
11. What is the difference between printf() and puts()?
12. What is the difference between getchar() and putchar()?
13. What is sprintf()?
14. What is sscanf()?
15. Why should gets() not be used?

---

47. Long/Descriptive Questions

Question 1

Explain formatted input and output functions in C with suitable examples.

Students should discuss:

- printf()
- scanf()

- format string
- conversion specifications
- &
- examples

Question 2
Explain different format specifiers used in C.
Students should cover:
%d
%u
%ld
%f
%lf
%Lf
%c
%s
%x
%o

Question 3
Explain field width, alignment and precision with suitable examples.
Example:
printf("%10d", 25);
printf("%-10d", 25);
printf("%10.2f", 25.678);

Question 4
Differentiate printf(), scanf(), sprintf() and sscanf().

Question 5
Explain character and string input/output functions in C.
Students should discuss:
getchar()
putchar()
fgets()
puts()
and explain why:
gets()
should not be used.

---

48. Practical Programming Exercises
I recommend giving BIT students these exercises in increasing difficulty.
Level 1 — Basic
1. Write a program to input two integers and display their sum.
2. Input a student's name, roll number and age and display them.
3. Input the radius of a circle and display its area.

Level 2 — Formatted Output
4. Display a student mark sheet using field width and precision.
5. Display the following table:
Roll Name Marks

101 Ram 85.50
102 Sita 91.25
103 Hari 78.75
Use appropriate formatting.

Level 3 — String Input
6. Input a student's full name using fgets() and display it using puts().
7. Write a program to input a full address containing spaces.

Level 4 — Integrated
8. Student Mark Sheet
Input:

- Roll number
- Name
- Five subject marks

Calculate:

- Total
- Average
- Percentage

Display a properly formatted mark sheet.

Level 5 — Reasoning
9. Debug the following program:

```
#include <stdio.h>

int main()
{
   int age;
   float salary;

   printf("Enter age and salary: ");
   scanf("%f %d", &age, &salary);

   printf("Age = %d\n", age);
   printf("Salary = %.2f\n", salary);

   return 0;
}
```

Ask students:
Identify all errors and explain why they are errors.
This is much better than simply asking them to memorize syntax.

---

49. Classroom Activity: "Predict the Output"
Give students:

```
#include <stdio.h>

int main()
{
   int a = 25;
   float b = 12.3456;
```

```
    printf("%d\n", a);
    printf("%10d\n", a);
    printf("%-10dEND\n", a);
    printf("%.2f\n", b);
    printf("%10.2f\n", b);

    return 0;
}
```

Before running the program, ask:

Predict the output and explain the purpose of each format specification.

Then run it.

This develops **reasoning rather than memorization**.

---

50. Classroom Activity: Think–Pair–Share

Question

Ask:

Why does scanf("%d", &age) require &age, while scanf("%s", name) normally does not require &name?

Think — 2 minutes

Students individually write their explanation.

Pair — 3 minutes

Students compare their answers.

Share — 5 minutes

Two or three pairs explain their reasoning.

Lecturer conclusion

Connect the answer to:

- memory address
- pointers
- arrays
- address operator
- function arguments

This creates a bridge from **Unit 3 I/O to pointers and arrays**, which is pedagogically valuable.

---

51. Key Takeaways for Students

Students should remember these **10 rules**:

1. printf() → formatted output.
2. scanf() → formatted input.
3. getchar() → read one character.
4. putchar() → output one character.
5. fgets() → safely read a line/string.
6. puts() → output a string with newline.
7. %d → integer.
8. %f → float in printf()/scanf() for float.
9. %lf → double in scanf().
10. Never use gets() in modern C.

And the most important conceptual rule:

printf() needs values to display; scanf() generally needs addresses where it can store input.
